MosaicOpt: An Algebraic Foundation for Agentic Memory Databases

ANONYMOUS AUTHOR(S)

Production LLM agents accumulate multi-modal memory across two timescales: long-term facts and preferences, and short-term working context. A memory unit typically carries text, one or more dense vector embeddings, structured metadata, and graph relations. Useful queries against this store are conjunctive expressions that mix all four modalities — for example, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and connected by a derivation link to a prior incident." The dominant engineering response, *multi-path recall* with application-level score fusion, has no principled semantics, no equivalence relation over plans, and breaks down on multi-vector conjunctions. We propose **MosaicOpt**, an *algebra-first* design that extends relational algebra with (i) a *materialization* operator μ that binds computed values (vector distances, BM25 scores) as first-class attributes, (ii) first-class vector, text, and graph predicates, and (iii) a graph expansion operator Γ. The optimizer is a Volcano/Cascades-style rule-based layer driven by an *equational theory* of seven foundational identities (M1–M5, G1, G2) over the extended operators, the assumed classical join-commutativity and join-associativity rules, derived corollaries (M4+M1, M5+G1, A4-vector + M1, A7-symmetry, A8-regrouping) and implied-by-definition rewrites (topK recognition, σ-τ commutation), combined with a cost-based layer that explicitly accounts for approximation. We prove the identities, give counterexamples for the five non-identities (N1–N5) that classical intuition suggests but approximation or graph-traversal asymmetry breaks, and sketch a prototype and evaluation plan. The argument is foundational: agentic memory is a query problem, query problems benefit from algebraic foundations, and the right foundation is an extension of relational algebra.

Additional Key Words and Phrases: agentic memory, relational algebra, query optimization, vector search, hybrid retrieval, rule-based optimizer, equational theory

ACM Reference Format:
Anonymous Author(s). 2026. MosaicOpt: An Algebraic Foundation for Agentic Memory Databases. In *Proceedings of ACM SIGMOD/PODS International Conference on Management of Data / Principles of Database Systems (PODS '27).* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction & Motivation

Large language model (LLM) agents are no longer stateless request–response loops. Production agents — coding copilots, personal assistants, autonomous research workers, multi-step planners — accumulate state across two timescales: *long-term memory*, which persists facts, preferences, and episodic traces across sessions; and *short-term memory*, which captures the working context of an in-flight session, including tool-call history and intermediate reasoning artifacts. This split is widely accepted in the agentic-systems literature, but its consequences for the storage layer have received less attention.

The content of agentic memory is fundamentally *multi-modal*. A single "memory unit" may carry: (i) raw text (a chat snippet, a code diff, a document excerpt); (ii) one or more dense vector embeddings, often produced by different encoders for different retrieval objectives (e.g. semantic similarity, intent classification, code search); (iii) structured metadata (timestamps, source, provenance, user tags, confidence scores); and (iv) graph structure (entity relations such as `derived_from`, `cites`, `contradicts`, `member_of`). A useful query against this store is seldom a pure similarity lookup; it is a *conjunctive* expression that mixes all four modalities. "Find passages about a fault tolerance

PODS '27,
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

bug from the last 30 days, semantically similar to this stack trace, attributed to a known subsystem, and connected by `derived_from` to a previous incident" is representative.

The dominant engineering response is *multi-path recall*: a fan-out across specialized engines (a vector index, a full-text index, a key–value store, a graph store), each producing a ranked candidate list, followed by application-level score fusion and re-ranking. Frameworks such as Mem0, Letta, Zep, and Cognee instantiate variations of this pattern. It works, and there is no shortage of impressive product demonstrations. But the pattern has four structural weaknesses:

(1) **No principled semantics.** The semantics of a fused query is defined by the application code that combines the per-channel scores. Two implementations of "find similar memory" in the same product can have silently different meanings.
(2) **Heuristic optimization only.** Without an algebra, there is no equivalence relation over plans. The "optimizer" is a hand-tuned orchestration pipeline whose rewrites are not derived from a sound theory.
(3) **Fusion policy leaks into the substrate.** Score combination functions (e.g. weighted sum, reciprocal-rank fusion, learned rerankers) are application concerns; once baked into a query template, they are hard to change without re-engineering the data path.
(4) **Top-k per column breaks conjunctive intent.** Each vector column is typically searched independently with its own top-k, then intersected. For multi-vector memories, the intersection of per-column top-k sets is frequently empty or impoverished, because the columns are correlated, not redundant.

Our position is that these problems are not implementation details; they are symptoms of a missing foundation. **MosaicOpt** proposes an *algebra-first* design in which relational algebra is extended with a small set of operators that make multi-modal memory queries first-class, and in which the resulting algebra — not application code — is the substrate contract between query and execution. The optimizer becomes a *rule-based* layer driven by an equational theory (Section 3), with a *cost-based* layer (Section 4) choosing between physical realizations that may legitimately differ in approximation behaviour.

Figure 1 shows the resulting architecture. The *application* layer (top, cream) holds the fusion policy as SQL `ORDER BY` and `LIMIT` constructs; the fusion weights live entirely in the SQL and the substrate has no opinion on them. The *logical algebra* (middle, white) is the substrate contract: a small set of operators over which the equational theory is stated, including the new μ (materialization) and Γ (graph expansion) operators. The *physical algebra* (bottom, light blue) realizes each logical operator through a concrete access path — HNSW, IVF, or brute-force scan for vector selection; B-tree for structured range; inverted index for BM25; CSR / adjacency map for graph walks. The ANN family bridge $H_{V,q,\tau}$ (bottom of the figure) is the point at which approximation enters the system — a fact the logical layer hides and the cost model (Section 4) makes explicit.

1.1 Why algebraic foundation?

Codd's relational algebra [2] and Selinger's cost-based optimizer [3] established that query processing benefits from a formal language for what is computable and a formal theory for what transformations preserve semantics. The Volcano/Cascades line further showed that the algebra is the *API*: a new operator ships with its equivalence rules and cost model, and the optimizer gains new capabilities without any central rewrite. **An algebra is not an implementation detail; it is the contract between query and execution.** When the algebra is rigorous, every query language is a syntactic surface over it, every optimizer decision is auditable, and every physical innovation is a drop-in.

Formally, an algebra provides four things that no collection of indexes and APIs can:

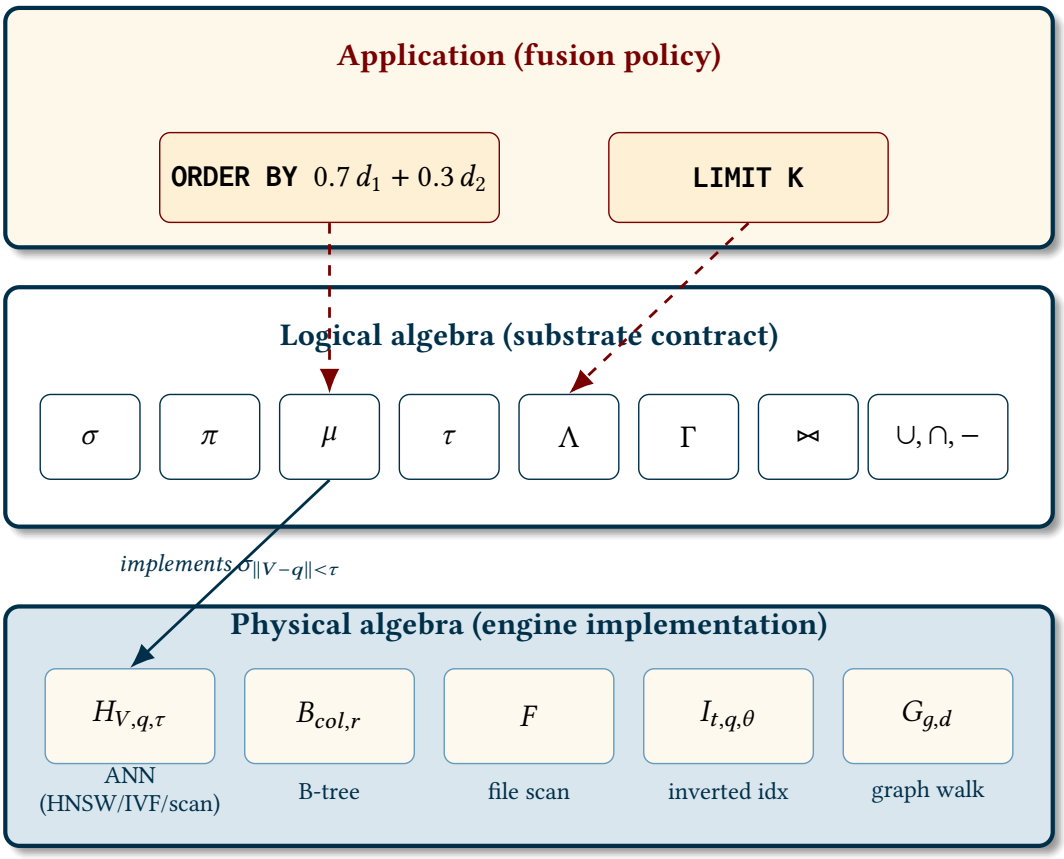


Fig. 1. Three-layer architecture of MosaicOpt.

- **A language for expressive power** — a precise definition of what is computable in the system, and what is not.
- **An equational theory** — a body of identities that govern when one query plan is equivalent to another.
- **A cost-model framework** — the substrate on which cost-based optimization rests, parameterized by the same logical operators the user wrote.
- **Composability** — new operators can be added without breaking old ones, because the algebra is the integration point. The Cascades maxim — *the algebra is the API* — makes extensibility a property of the foundation, not of the optimizer code.

Agentic memory is the *worst case* for a "bag of engines" approach: it spans two timescales (long-term and short-term) with different access patterns and consistency guarantees; its content is genuinely multi-modal (text, multiple vector columns, structured metadata, graph relations on the same memory unit); and the queries that matter are *conjunctive across modalities* — a vector range intersected with a metadata filter joined with a graph reachability predicate. The field shows *convergent evolution*: every Agentic framework (Mem0, Letta, Zep, Cognee) has independently reinvented the same patterns (multi-path recall, score fusion, re-ranking) with no shared vocabulary, type system, or notion of plan equivalence. The practical consequence is that each framework has

a different *memory API* with no formal semantics, and porting a workload requires reading source code rather than reasoning from a specification.

1.2 Why an algebraic foundation helps

The algebra gives database researchers and engineers a precise substrate on which to develop new query patterns and to *prove* their correctness. A new materialization pattern or join algorithm becomes a new operator with a documented equational theory; its rewrite rules are *derived* from the algebra, not guessed from benchmarks. The optimizer's plan space becomes *derivable*: each new operator is added with its identities and the optimizer gains the corresponding rewrites "for free". The algebra is the *common currency* between application and engine — an application written once is portable across engines that share the algebra, and an engine innovation is portable to every application that uses the affected operator.

1.3 Data substrate for Agentic applications

Four use cases illustrate the substrate requirements: **episodic recall** (conjunctive vector + metadata + graph queries); **cross-session fact retrieval** (multi-vector recall with structured filters); **working context assembly** (fast filtered recall by recency and entity); **tool-trace analysis** (graph queries over `derived_from` and `follows` edges, joined with vector similarity on tool outputs). All four are *query problems* expressible as expressions over a memory store and amenable to the same optimizer pipeline. With MosaicOpt's algebra, the application writes its query in SQL (Section 4); the substrate applies M1-style rewrites to fuse materialization with selection, M5/G1/G2 to push operators through graph expansion, M4/M5 to push materialization through joins, and CBO physical-operator selection. The application does not need to know about HNSW/IVF/scan, B-tree, or inverted indexes — these are hidden as physical realizations of a single logical operator; the application policy (fusion weights, thresholds, top-k) stays in the SQL.

1.4 Contributions

This paper makes two contributions:

- **An extended relational algebra for agentic memory** (Section 2): a *materialization* operator μ, first-class vector/text/graph predicates, an explicit *graph expansion* Γ, and a strict logical/physical separation under bag semantics.
- **An equational theory with proofs** (Section 3): seven *foundational* identities (M1–M5, G1, G2) over the extended operators, the assumed classical A1–A8 (including join commutativity and associativity; A1, A4 extend naturally to vector predicates), the derived chains M4+M1, M5+G1, A4-vector+M1, the classical applications A7-sym and A8-regroup, the definitional τ-Λ-topK rewrite, the row-preserving σ-τ commutation, five non-identities (N1–N5) that approximation, graph-traversal asymmetry, or the traversal-vs-origin column split break, and a confluent RBO (Theorem 13).

The paper is intentionally foundational. The algebra is the *common currency* between application and engine; a shared vocabulary will benefit the field just as it did in the relational world. The companion SIGMOD paper [1] develops the implementation — the SQL surface, the rule-based and cost-based optimizers, and the prototype — that honors the algebraic contract proved here.

2 Extended Algebra

2.1 Background

Relational algebra and the classical optimizer. The relational model [2] and its algebra remain the dominant formal substrate for declarative query languages; Codd showed that a small set of

operators (σ, π, product, union, difference, rename) is sufficient for any first-order query. The second pillar is **query optimization**: Selinger's cost-based System R optimizer [3] enumerated alternative orderings under a cardinality model; the **Volcano** [4] and **Cascades** [5] frameworks generalized this to a rule-based outer loop driving a cost-based inner search, with the algebra as the API. Two ideas from this tradition are central: an *equational theory* of plan identities is the foundation for principled rewriting, and *logical–physical separation* allows physical innovations to be plugged in without changing semantics.

Vector search and modern IR. Information retrieval contributed the *vector space model* [6] and BM25 [7] as principled relevance scoring functions. The neural era added dense embeddings [8, 9] and approximate nearest neighbour (ANN) indexes for sub-linear search: HNSW graphs [10] are the dominant production choice, with IVF inverted files and brute-force linear scan as alternatives. A crucial property of ANN indexes is that they are *approximate*: the result of an HNSW range query is not the exact set within distance τ but an approximation whose quality depends on the index parameters (`ef`/`M` for HNSW, `nlist`/`nprobe` for IVF). This approximation is invisible at the algebraic level but central to the cost model: a high-recall ANN plan costs more than a low-recall one, and the optimizer must trade recall against latency.

Hybrid query systems. The last decade has produced three lines of *hybrid* systems: **vector-aware RDBMS extensions** (`pgvector`, DuckDB `vss`, Oracle AI Vector Search) add a vector type and ANN index at the function-call level, with no first-class identities; **purpose-built hybrid stores** (Pinecone, Weaviate, Qdrant, Milvus, Vespa) are vector-first with metadata filtering bolted on; **multi-modal query languages** (SQL/MM, SPARQL with full-text, Cypher) mix graph, text, and (in a few cases) vector. The common pattern: every system fuses scores at the application layer rather than extending the algebra that defines what counts as an answer.

The gap. What is missing is an *equational* view of multi-modal memory queries in which vector distance, BM25 relevance, and graph reachability are first-class predicates on a par with $A = c$; in which computed values (distances, scores) are first-class attributes; and in which the boundary between the application's *fusion policy* and the substrate's *evaluation strategy* is explicit and contractually enforced. That is the gap MosaicOpt addresses.

We extend the classical relational algebra with three new kinds of construct: a *materialization* operator μ that lifts computed values to first-class attributes, first-class multi-modal predicates (vector distance, BM25, graph reachability), and a *graph expansion* operator Γ. Bag semantics are used throughout, matching SQL and avoiding the row-collapse anomalies of set semantics in the presence of duplicate memory units; the logical/physical separation is enforced (physical operators are introduced in Section 4 and never appear here).

2.2 Logical/physical separation

Following System R and Cascades, we treat the *logical* algebra as the substrate contract — what the user asks for — and the *physical* algebra as the engine contract — how it is evaluated. Equivalences hold in the logical algebra; physical operators are *implementations* of logical operators, and a logical equivalence does not in general imply that two physical realizations are interchangeable (Section 3.4, N1–N3). This separation is what makes the system *composable*: the ANN family (HNSW, IVF, scan) can be added, swapped, or replaced without changing the language the user sees.

2.3 Operator inventory

A *logical plan* is a term over the following operators. R denotes a stored base relation. L, M denote logical plans. φ denotes a predicate, e a materialization expression, A a set of attribute names,

O an ordering expression, K a non-negative integer, c a fresh attribute name, g a graph rule (a labelled-edge pattern), and d a non-negative integer depth.

The logical operators are: **base relation** R; **selection** $\sigma_\varphi(L)$ (rows satisfying φ); **projection** $\pi_A(L)$; **materialization** $\mu_{e\to c}(L)$ (per row, compute e, bind to fresh c); **join** $L \bowtie_\psi M$; **set ops** $L \cup M$, $L \cap M$, $L - M$; **rename** $\rho_{a\to b}(L)$; **ordering** $\tau_O(L)$ and **limit** $\Lambda_K(L)$; the abbreviation $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L))$ (the K smallest rows of L under O, a primitive we single out because the CBO gives it a dedicated physical realization, Section 4); and the new **graph expansion** $\Gamma_{g,d}(L)$ (from a node-id attribute, traverse rule g to depth d).

Definition 1 (Bag semantics)**.** A relation is a multiset of tuples. All operators are multiset-preserving: σ, μ, Γ preserve row multiplicity; projection preserves multiplicity; set operations combine multiplicities additively for $\cup$, by min for $\cap$, and by subtraction (clamped at 0) for $-$. τ_O produces a totally ordered sequence; Λ_K truncates it.

Bag semantics was chosen because the substrate must be able to express "the same memory unit appears N times" (e.g. an episodic trace where the same event is recorded with several slightly different timestamps), and because it matches the SQL semantics that application code already expects.

2.4 Predicate forms

A predicate φ is a Boolean combination of atoms. The atom forms are: (i) *classical* ($A = c$, $A < c$, $A > c$, $A \in S$); (ii) *vector* ($\|V_a - q\| < \tau$ where V_a is a vector attribute, q a query-time bound vector, and $\tau \geq 0$ a threshold); (iii) *text* ($\text{BM25}(t, q) > \theta$ with parameters (k_1, b) from [7]); (iv) *graph* ($\text{reachable}(u, v)$, $\text{edge}(u, v, \ell)$); and (v) *Boolean combinations* ($\wedge$, $\vee$, $\neg$). *Query-time bound parameters* (q, τ, θ) are first-class values distinct from input attributes, making predicates safely late-binding: the same algebra expression serves all query instances, only the bound parameters change.

2.5 Materialization: motivation, expressions, and semantics

Inlining per-column distances in `ORDER BY` (e.g. `ORDER BY` $\|V_a - q_a\| + \|V_b - q_b\|$) is permitted by SQL extensions, but it forces the application to write the distance expression everywhere it is used and leaves the optimizer no room to share computation between a `WHERE` filter and an `ORDER BY` rank. The materialization operator $\mu_{e\to c}(L)$ binds a per-row expression e to a fresh attribute name c, making the materialized value referenceable by downstream σ, π, τ, and μ (see Section 2.7 for an example). Valid forms of e are classical ($A + B$, $f(A)$), vector ($\|V_a - q\|$), text ($\text{BM25}(t, q)$), and Boolean or arithmetic combinations. The fresh-name condition (Def. 2) requires c to be absent from $\text{schema}(L)$; ρ is provided to free names.

Definition 2 (Fresh-name condition)**.** c in $\mu_{e\to c}(L)$ must not already appear in $\text{schema}(L)$. ρ is provided to free names if needed.

The semantics is the multiset $\{\, r \oplus \{c \mapsto e(r)\} : r \in L \,\}$. If $e(r)$ is undefined (e.g. distance to a null vector), the row is dropped — consistent with SQL's `NULL` handling and essential for safety in the presence of missing embeddings.

2.6 Graph expansion

$\Gamma_{g,d}(L)$ takes a plan whose rows contain a node-id attribute (u, say) and a graph attribute, applies graph rule g up to depth d, and returns the *target* nodes of those traversals (joined with the originating row). The depth is inclusive of intermediate hops. g is a labelled-edge pattern: a regular expression over edge labels, evaluated against the graph. The graph itself is supplied as a relation-valued attribute; the expansion is a join between L and the traversal result.

Γ is intentionally narrow. We do not attempt to encode path queries as selection predicates; this keeps the operator set small and the rewrite theory tractable (Section 3, G1, G2). Path information can be recovered by a μ that records the path taken.

Remark 1 (Γ identity at depth 0). $\Gamma_{g,0}(L) = L$ for any rule g and any plan L — at depth zero no traversal is performed, so the expansion is the identity on rows. The rule g is irrelevant at $d = 0$. This is the unit of Γ composition under the G2 transitivity rule of Section 3.2: for any d_1, d_2, $\Gamma_{g,d_1}(\Gamma_{g,d_2}(L)) = \Gamma_{g,d_1+d_2}(L)$, and the $d = 0$ case is the trivial instance $\Gamma_{g,0}(L) = L$.

2.7 Example queries

We use a single running example throughout. Let *Mem* be a memory relation with schema *Mem*(*id*, *text*, *ts*, ***emb***, ***emb_alt***, *user*, *subsystem*) and *KG* a graph relation *KG*(*src*, *dst*, *label*). The query "find similar passages in the fault-tolerance subsystem from the last 30 days, ranked by a weighted sum of two distances, top 10" is

$$
\begin{aligned}
Q_1 &= \mathit{Mem},\\
Q_2 &= \sigma_{\mathit{subsystem}=\text{'ft'} \wedge \mathit{ts} > \text{now}()-30d}(Q_1),\\
Q_3 &= \mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\| \to d_1}\ \mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\| \to d_2}(Q_2),\\
Q_4 &= \sigma_{d_1 < \tau_1 \wedge d_2 < \tau_2}(Q_3),\\
Q_5 &= \tau_{0.7 \cdot d_1 + 0.3 \cdot d_2}(Q_4),\\
Q_6 &= \Lambda_{10}(Q_5).
\end{aligned}
$$

The algebra's job is to make this expression *rewriteable* — to compute d_1, d_2 only on rows that survive the filter (M1) and to enable top-K early termination (the $\tau\,\Lambda$ / topK recognition in Section 3.5). A graph-aware variant intersects the result with a two-hop Γ expansion $R_2 = \Gamma_{\mathit{label}=\text{'derived_from'}, d=2}(R_1)$.

3 Equational Theory

The optimizer is driven by an *equational theory* of identities over logical plans, together with the *non-identities* that classical intuition suggests but approximation, graph-traversal asymmetry, or the traversal-vs-origin column split break — so the optimizer knows not to apply them. This section unifies both: the *seven* genuinely-new foundational identities that the extended operators license (M1–M5, G1, G2), the assumed classical identities that the RBO uses for join reordering (A7, A8, alongside A1–A6; A1, A4 apply to vector predicates by substitution), the derived chains and classical applications (M4+M1, M5+G1, A4-vector+M1, A7-sym, A8-regroup), the implied-by-definition $\tau\,\Lambda$ / topK and σ-τ rules, the non-identities (N1–N5) with counterexamples, and the convergent rewriting analysis.

3.1 Overview

The theory plays two roles. *First*, it is the foundation for the RBO (Section 4): every rewrite is justified by an identity (M1–M5, G1, G2, or a derived combination M4+M1, M5+G1, A4-vector+M1, A7-symmetry, A8-regrouping, or the implied σ-τ rewrite and $\tau\,\Lambda$ / topK recognitions), so the optimizer's behaviour is auditable. *Second*, the non-identities (N1–N5) discipline the optimizer against rewrites that hold in classical relational algebra but break under approximation, asymmetric physical costs, or graph-traversal semantics. The cost model is responsible for the physical cost of every rewrite; the equational theory is responsible for its *correctness*. Both are first-class data: the RBO loads them, the cost model consults them, and the executor relies on them for safety of adaptive re-planning.

3.2 Foundational axioms

The seven identities below are the *new* algebraic content licensed by the extended operators; each is stated with its applicability conditions and a pointer to the proof (full proofs in Appendix A.1). We do *not* state as separate axioms the identities that are *implied* by other operators or by classical RA: classical join commutativity (A7) and join associativity (A8) are assumed (they are in every RA textbook, and the RBO uses them as join-reordering moves to enumerate the $O(n!)$ join orderings of Section 4); the τ / Λ / topK family is closed by the definition $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L))$ from Section 2 and the σ-τ commutation follows immediately from the row-preserving semantics of σ and τ (Def. 1). A handful of other classical identities (selection through intersection/difference, projection through set operations, join through set operations) are likewise assumed.

Theorem 1 (M1 — Selection commutes with materialization on the materialized column)**.** *Where c is a fresh attribute name and e is an expression depending only on attributes of L:*

$$\sigma_{c<\tau}\, \mu_{e\to c}(L) \;=\; \mu_{e\to c}\, \sigma_{e<\tau}(L).$$

(*Proof in Appendix A.1.*)

Theorem 2 (M2 — Materialization is commutative)**.** *For fresh names c_1, c_2 and expressions e_1, e_2 depending only on L's attributes (and not on each other's outputs):*

$$\mu_{e_1\to c_1}\, \mu_{e_2\to c_2}(L) \;=\; \mu_{e_2\to c_2}\, \mu_{e_1\to c_1}(L).$$

(*Proof in Appendix A.1.*)

Theorem 3 (M3 — Materialization through projection)**.** *For fresh c and expression e depending only on attributes in A:*

$$\mu_{e\to c}\, \pi_A(L) \;=\; \pi_{A\cup\{c\}}\, \mu_{e\to c}(L).$$

(*Proof in Appendix A.1.*)

Theorem 4 (M4 — Materialization pushdown through join)**.** *Where e depends only on attributes of L and c is a fresh attribute name:*

$$\mu_{e\to c}\, (L \bowtie_\psi M) \;=\; (\mu_{e\to c}(L)) \bowtie_\psi M.$$

(*Proof in Appendix A.1.*)

Theorem 5 (M5 — Materialization pushdown through graph expansion)**.** *Where e depends only on attributes of L, c is fresh and is not the graph rule's node-id attribute (Def. 2):*

$$\mu_{e\to c}\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \mu_{e\to c}(L).$$

This is the μ-counterpart of G1 and the rewrite the RBO generates for graph queries that materialize a per-origin attribute (e.g. a distance to a fixed query point) and then expand. The two sides produce the same multiset of (r, v, c) tuples; the cost differs because the LHS computes e on every traversal row while the RHS computes e on every origin row. When $|\Gamma_{g,d}(L)| > |L|$ (the typical case for $d \geq 1$ and a non-trivial graph), the RHS is cheaper, and the CBO picks it.

(*Proof in Appendix A.1.*)

Theorem 6 (G1 — Selection pushdown through graph expansion)**.** *Where φ references only attributes of L (and not attributes of the graph rule's traversal result):*

$$\sigma_\varphi\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \sigma_\varphi(L).$$

(*Proof in Appendix A.1.*)

Theorem 7 (G2 — Graph expansion is transitive). *Let g_1, g_2 be graph rules over the same graph attribute with disjoint label sets. If g_1's traversal-result attribute and g_2's source attribute are disjoint (no intermediate node of g_2 is a valid origin of g_1), then the composition $g_1; g_2$ satisfies:*

$$\Gamma_{g_1,d_1}\,\Gamma_{g_2,d_2}(L) \;=\; \Gamma_{g_1;g_2,\,d_1+d_2}(L).$$

G2 lets the RBO collapse nested Γs into a single expansion at depth $d_1 + d_2$ — "two `derived_from` *hops" becomes "one* `derived_from ; derived_from` *hop".*

(*Proof in Appendix A.1.*)

3.3 Derived identities

Composing the foundational axioms and the assumed classical identities yields additional rewrites:

- **A4-vector (in code: J1).** A4 specialized to a vector predicate — $\sigma_{\|V_a-q\|<\tau}(L \bowtie_\psi M) = (\sigma_{\|V_a-q\|<\tau}(L)) \bowtie_\psi M$ when $V_a \in \text{schema}(L)$ — is what the CBO applies on the conjunctive multi-vector case. A1 likewise specializes to two vector predicates on distinct columns. Neither is a new algebraic identity — both follow from A1/A4 by predicate substitution — but the codebase keeps them as named redex patterns (V1, J1) so the RBO can match them explicitly.
- **A7-symmetry (reverse join).** A7 in the reverse direction; the CBO chooses the cheaper side per Theorem 9.
- **A8-regrouping (different join parenthesization).** A8 enumerates the $O(n!)$ join orderings; the CBO picks the best. (For two joins, A8 is degenerate; for $n \geq 3$ it unlocks the full plan space.)

A4-vector+M1 pushes a join-side vector selection below a materialization binding the same distance (apply A4 then M1). **M4+M1** reorders a join-side materialization so the expensive per-row computation runs only on rows that survive the join's downstream filter (apply M4 then M1) — the converse of A4-vector+M1 and the rewrite that powers the Section 4 running example. **M5+G1** reorders a graph query with both row-level filter and materialization (apply G1 then M5). **topK** and the σ-τ commutation are both *implied-by-operator-definition* rules: topK by $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L))$ from Section 2, and σ-τ commutation by the row-preserving semantics of σ and τ (Def. 1); the RBO applies them in the zero-weight post-w rewriting pass. The algebra is closed under composition — A1–A8 plus the seven foundational axioms plus the rewrite rules above suffice to cover the optimization of every plan whose operators draw from the Section 2 inventory.

3.4 Non-identities and counterexamples

Some rules that look natural in a relational setting **do not hold** in the extended algebra. We document these explicitly so the optimizer knows not to apply them. The same logical plan in two realizations can have different outputs; the optimizer must treat this as a *cost* phenomenon, not a *correctness* phenomenon.

3.4.1 N1: Approximation breaks logical equivalence.

Theorem 8 (N1 — ANN approximation breaks logical equivalence). *Logical $\sigma_{\|V-q\|<\tau}(R)$ is exactly defined; physical $H_{V,q,\tau}(R)$ under HNSW or IVF is approximate (misses true neighbours, may include non-neighbours). A logical equivalence does not imply that two physical realizations preserve it.*

Counterexample 1. A row with $\|\boldsymbol{v} - \boldsymbol{q}\| = 0.99\tau$ may be missed by a small-`ef` HNSW but returned by an exact linear scan: two valid implementations of the same logical selection disagree. The optimizer must model approximation as a first-class *cost dimension* with quantified recall/precision.

3.4.2 N2: Similarity join asymmetry.

Theorem 9 (N2 — Similarity join asymmetry). *$R \bowtie_{\|V_a - V_b\| < \tau} S$ and $S \bowtie_{\|V_b - V_a\| < \tau} R$ are logically equivalent; the two physical sides are not interchangeable in cost.*

Counterexample 2. Probing a 10^6-row HNSW on R with each row of a 10^3-row S costs 10^3 probes; the reverse (probe S for each row of R) costs 10^6 — three orders of magnitude apart. The RBO generates both orderings; only the CBO can decide which is cheaper.

3.4.3 N3: Limit and order do not distribute over union.

Theorem 10 (N3 — Limit and order do not distribute over union). *$\Lambda_K\, \tau_O\, (R \cup S) \;\neq\; \Lambda_K\, \tau_O(R) \cup \Lambda_K\, \tau_O(S)$ in general.*

Counterexample 3. Let R and S each contain 100 rows with a single scoring column; draw R.score $\sim \mathcal{U}([0,1])$ and S.score $\sim \mathcal{U}([9,10])$. The LHS $\Lambda_5\, \tau\, (R \cup S)$ returns the 5 rows with the *globally* smallest scores — all from R, with scores in $[0, 0.05]$. The RHS $\Lambda_5\, \tau(R) \cup \Lambda_5\, \tau(S)$ returns the 5 smallest of R *plus* the 5 smallest of S — 10 rows total, spanning the disjoint score ranges. The identity would be useful for distributed top-K (each shard returns its local top-K then unions), but does not hold; the distributed top-K problem is precisely about Section 5.3's threshold-exchange protocol.

3.4.4 N4: Materialization through graph expansion fails on traversal columns.

Theorem 11 (N4 — μ through Γ on the traversal column). *M5 (Theorem 5) requires e to depend only on* origin *attributes of L. When e references the traversal-result attribute produced by Γ, μ does* not *commute with Γ:*

$$\mu_{e \to c}\, \Gamma_{g,d}(L) \;\neq\; \Gamma_{g,d}\, \mu_{e \to c}(L)$$

in general when e depends on the traversal result.

Counterexample 4. Let L have rows $(id, \boldsymbol{emb})$ and let $\Gamma_{\text{follows},1}$ produce an additional $\boldsymbol{followed_emb}$ attribute. Take $e := \|\boldsymbol{followed_emb} - \boldsymbol{q}\|$, a distance to the *followed* node's embedding. The LHS $\mu_{e \to c}\, \Gamma(L)$ materializes c against the followed-node vector; the RHS $\Gamma_{\text{follows},1}\, \mu_{e \to c}(L)$ materializes $c' = \|\boldsymbol{emb} - \boldsymbol{q}\|$ against the *origin* vector (the followed-attribute is not yet bound) and then expands — the two multisets compute different distances row-by-row. N4 blocks the RBO from applying M5 when the materialization expression reads a traversal-attribute column.

3.4.5 N5: Limit and order do not commute with graph expansion.

Theorem 12 (N5 — Λ/τ do not commute with Γ). *$\Lambda_K\, \tau_O\, \Gamma_{g,d}(L) \;\neq\; \Gamma_{g,d}\, \Lambda_K\, \tau_O(L)$ in general.*

Counterexample 5. Let L have 1000 origin rows with a *score* column and let $\Gamma_{\text{cites},1}$ expand each origin to a variable $1 \le n_i \le 50$ of citing nodes. The LHS $\Lambda_{10}\, \tau_{score}(\Gamma(L))$ returns the 10 rows of the expansion with the globally smallest *score* (some origins may contribute several winners; some origins none). The RHS $\Gamma(\Lambda_{10}\, \tau_{score}(L))$ first picks the 10 origin rows with the smallest *score* and then expands each — returning up to $10 \cdot 50 = 500$ rows that all share those 10 origins. The two multisets differ in cardinality and in content: the LHS asks "across all reachable nodes, which K score lowest?"; the RHS asks "among the K highest-scoring origins, which nodes are reachable?". N5 blocks the RBO from pushing Λ or τ through Γ; the cost model of Section 4 reflects this by giving $\tau\, \Lambda\, \Gamma$ a substantially higher cost than $\Gamma\, \tau\, \Lambda$ when the expansion fan-out is large.

3.5 Convergent rewriting

The equational theory can be oriented into a **rewriting system** $\to_{\text{RBO}}$ on logical plans; the optimizer of Section 4 is an interpreter for $\to_{\text{RBO}}$. Define the weight $w(P) = \alpha s(P) + \beta r(P) + \gamma p(P)$ where s counts σs on μ-bound attributes (M1-fixable), r counts redundant μs, and p counts πs that drop needed attributes (M3-fixable). **M1, M3, M4, G1 strictly decrease** w, and **M2, M5, G2, and the implied** σ-τ **reordering are zero-weight rewrites**: M2 is a μ-reordering that preserves s, r, p and is bounded by the $k!$ orderings on k materializations; M5 is a μ-Γ reordering that preserves s, r, p and is bounded by the number of μ-Γ inversions in P; **G2 is a** Γ-Γ **collapse** that preserves s, r, p and is bounded by the number of nested-Γ inversions in P (one collapse per nesting, hence the depth d of the result); the σ-τ rewrite (implied by the row-preserving semantics of σ and τ) preserves s, r, p and is bounded by the number of σ-τ inversions in P. The M1/M3/M4/G1 sub-rewriting system terminates by the strict decrease of w; once w reaches a local minimum under those four rules, only the zero-weight rewrites remain, and each terminates by its bounded inversion count. The CBO-applied topK recognition ($\tau_O \Lambda_K \leftrightarrow \text{topK}_O$ per $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L))$) is a definitional rewrite that the CBO applies as a physical-realization choice; it is in one-to-one correspondence with its inverse and so trivially terminates. The critical pairs from overlapping redexes (M1∩M1, M1∩M4, M1∩G1, M1∩M5, M4∩G1, M4∩M5, M5∩G1, G1∩A4, G2∩G2, G2∩G1, G2∩M5, M2∩M1, M2∩M5, σ-τ ∩ M1, σ-τ ∩ M3) all join by further rewrites, giving local confluence. (The A7/A8 classical join-reordering identities and the $\tau\Lambda$ / topK definitional rewrite are included for completeness; in practice the CBO applies the $\tau\Lambda$ / topK rewrite directly as a physical realization choice, and the RBO applies A7/A8 as join-reordering moves.) By Newman's lemma [13], termination + local confluence imply confluence and a unique normal form:

Theorem 13 (RBO Confluence)**.** *For any starting logical plan P and any two RBO rewrite sequences $P \to^*_{\text{RBO}} P^*$ and $P \to^*_{\text{RBO}} P^{**}$, there exists a plan P^∞ with $P^* \to^*_{\text{RBO}} P^\infty$ and $P^{**} \to^*_{\text{RBO}} P^\infty$. The normal form is unique: it depends only on P, not on rewrite order.*

(*Proof in Appendix A.2.*)

Theorem 13 is the formal correctness guarantee of the RBO — the optimizer can be implemented as a simple greedy loop (apply any rule that decreases w) without losing correctness. The CBO inherits confluence "for free": its search is over the unique canonical form, so its choice of physical realization is well-defined and reproducible.

4 Implementation: A Brief Note

The equational theory of Section 3 is the foundation on which an implementation rests. The companion SIGMOD paper [1] develops the full implementation: the SQL/MM-style surface that desugars into the algebra, the rule-based optimizer that applies the equational rewrites, the cost-based physical-operator selector, and the prototype built on DuckDB, `faiss-cpu`, and `rank_bm25`. The present paper restricts itself to the algebraic content; we record here only the design choices that *constrain* the equational theory (rather than describe the implementation in detail).

Logical/physical separation. The cost model and physical operator catalog are *separate* from the equational theory. The identities hold for any choice of physical realization; the CBO picks a realization from a precomputed physical catalog. The cost model is a 4-tuple $(io, cpu, mem, approx) \in \mathbb{R}^4_{\geq 0}$, the fourth component quantifying the recall penalty of an approximate ANN strategy. The cost model is used to *search* the plan space, not to *define* the equivalence class.

TopK recognition. The $\tau\Lambda$ family is closed by the definition $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L))$; the rewrite $\tau_O \Lambda_K \leftrightarrow \text{topK}_O$ is applied by the CBO as a physical-realization choice, not as a logical

RBO rewrite. This definitional rewriting is one-to-one with its inverse, so it trivially terminates; the RBO's confluent ARS does not depend on it.

SQL surface. The SQL/MM-style surface — `IS NEAR`, `MATCHES`, `IS REACHABLE` — is the application's primary interface. Its desugaring into the algebra is total and semantics-preserving, so every SQL query expresses a plan in the algebra, and every plan in the algebra has a SQL rendering. The design rationale (downstream of this paper's algebraic content) is that the SQL extension must read as a *first-class* language construct, not as a function-style abbreviation, so the substrate is committed to understanding the construct and to applying the equational theory to it. The companion SIGMOD paper develops the desugaring rules in full and reports the four experiments that validate the end-to-end pipeline.

Boundary with the implementation. The implementation is open-source and is the primary artifact that the SIGMOD paper analyzes. The equational theory in the present paper is the *contract* that the implementation honors: every SQL query routed through the optimizer must produce a result equal (as a multiset) to the result the algebra evaluates, and the optimizer's rewrite sequence must terminate at a unique normal form (Theorem 13).

5 Discussion

5.1 Related work

We identify the lineage of MosaicOpt precisely — the goal is not to enumerate all related work but to make clear what is novel.

Classical relational algebra. MosaicOpt inherits from Codd's relational algebra [2] (the small operator set that expresses any first-order query, refined over time and crystallized in SQL) and from the Selinger/Volcano/Cascades line [3–5] (cost-based optimization, extensible rule-based rewriters, the maxim that the algebra is the API). These two pillars are unchanged; what MosaicOpt adds is (i) a set of new identities (M1–M5, G1, G2) for the extended operators, and (ii) an explicit convergent-rewriting analysis (Section 3.5).

Algebraic extensions of the relational model. MosaicOpt continues a long lineage of "extend the algebra, prove identities" work: **nested** relational algebra [20] adds set-valued attributes (MosaicOpt adds first-class materialization); **object-relational** extensions [21] add user-defined types; **temporal** algebras [22, 23] add time-aware predicates; **semi-structured** algebras [24] add tree-structured data and bisimulation-style identities. MosaicOpt extends in a *new direction* — multi-modal (text + vector + graph) in the context of agentic memory — and provides, in one place, a SQL surface and a proven equational theory the prior extensions did not.

Equational theories in query optimization. The classical identities (selection pushdown, join commutativity, projection pushdown) have been in the optimizer's arsenal since System R. The convergent-rewriting analysis of Section 3.5 draws on Newman's lemma [13], Huet's critical-pair analysis [14], and the Terese textbook [15]. The closest database prior is Atzeni and Torlone's [25] algebraic treatment of schema and view integration across heterogeneous data models, whose algebraic-rewriting pattern is identical. MosaicOpt's analysis is, to our knowledge, the first such analysis for an algebra that includes vector, text, and graph operators.

5.2 Hybrid query algebra (recent)

The last decade has produced a sustained effort to bring vector and relational together, but with very few *theoretical* contributions. The systems are numerous; the algebras are few: **vector-aware RDBMS extensions** (PostgreSQL `pgvector`, DuckDB `vss`, Oracle AI Vector Search) add a vector

type, an ANN index, and distance predicates at the function-call level — systems, not theories; **purpose-built hybrid stores** (Pinecone, Weaviate, Qdrant, Milvus, Vespa) are vector-first with metadata filtering bolted on and expose no named identities; **multi-modal query languages** (SQL/MM, SPARQL with full-text, Cypher) mix graph, text, and (in a few cases) vector, but SPARQL's vector fusion remains open. The common pattern: every system fuses scores at the application layer. MosaicOpt's contribution is to make the fusion *first-class*: a materialization expression is a syntactic construct, the optimizer sees it, the cost model accounts for it, and the equational theory of Section 3 applies.

5.3 Open theoretical problems

Each layer of the algebra, equational theory, and optimizer opens new theoretical problems that the algebra makes accessible but does not yet solve. We enumerate the most directly consequential.

Complexity of CBO.. Classical join ordering is solvable in $O(n^2)$ for left-deep trees [3] but NP-hard for bushy trees [18, 19]. When the cost model also ranges over probabilistic predicates, the problem becomes strictly harder still [12]. The CBO of Section 4 adds three new dimensions: (i) *vector selections* as first-class operators contribute a *continuous* (`ef-`, `nprobe`-parameterized) choice, not a discrete one; (ii) *graph expansion* has a direction-parameterized realization whose cost depends on *graph density* at the expansion point; (iii) *materialization sharing* is a partition problem — each μ may be shared with another μ or kept distinct, and the CBO must decide. We conjecture NP-hardness in the general case and conjecture polynomial-time subclasses for queries that bound the number of graph expansions, the depth of materialization sharing, and the cardinality of physical realizations per operator. Identifying these subclasses — and proving the tractable/intractable boundary — is a central open problem.

Adaptive optimization. The cost model is static — evaluated once at planning time — but wrong in practice (cardinality drifts, ANN recall varies with the query, BM25 selectivity is hard to predict). *Adaptive optimization* [4] must interact correctly with the algebra's convergent rewriting: the re-planned logical plan must be a "refinement" of the original, and any non-normal-form subplan the CBO switches to must later re-rewrite — a property we conjecture holds locally to the switched subtree.

Distributed execution. A distributed extension requires a network-cost model, partition-aware physical operators (e.g. a partition-aware TopK that respects N3), and an extension of the equational theory to reason about partitioning. The asymmetry is that graph expansion Γ is harder to parallelize than vector selection — a graph expansion must traverse edges across partition boundaries, while a vector selection is partitioned by the index itself. The cost of $\Gamma_{g,d}$ is a function of the partition structure, not just the graph size.

Write paths and memory evolution. The algebra is read-only. A write algebra must model (i) *incremental materialization updates* (when a base row changes, every μ that depends on it must be invalidated or recomputed; the equational theory gives the semantics, incremental maintenance is harder); (ii) *index updates* (ANN and inverted indexes must stay consistent with the base data); and (iii) *cost model drift* (cardinality and selectivity parameters drift and need re-calibration). Writes are the largest open problem in the paper.

5.4 Limitations and out-of-scope

- **No new physical data structures.** ANN indexes, B-trees, and inverted indexes are external; the contribution is the *algebraic* use of these primitives, not their internal design. A learned-index primitive [11] is a drop-in replacement in the bridge table.

- **No new learning-based optimization.** The MosaicOpt contribution is at the *logical* layer; a learned optimizer can *instantiate* the cost model without touching the algebra.
- **No new transaction model.** The algebra is read-only and single-node; transactional and distributed extensions are out of scope and addressed in the open problems above.

These are *scope choices* that keep the contribution focused on the algebraic foundation.

References

[1] Anonymous. The MosaicOpt system: An algebraic implementation of an agentic memory algebra (companion paper). *Under submission to SIGMOD*, 2026.

[2] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.

[3] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34, 1979.

[4] G. Graefe. Volcano — an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.

[5] G. Graefe. The Cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.

[6] G. Salton, A. Wong, and C. S. Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.

[7] S. Robertson, S. Walker, and S. Jones. Okapi at TREC-3. In *NIST Special Publication 500-225: Proceedings of the Third Text REtrieval Conference (TREC-3)*, 1995.

[8] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. In *ICLR Workshop*, 2013.

[9] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP*, 2019.

[10] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.

[11] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. The case for learned index structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*, pages 489–504, 2018.

[12] N. Dalvi and D. Suciu. Efficient query evaluation on probabilistic databases. *VLDB Journal*, 16(4):523–544, 2007.

[13] M. H. A. Newman. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics*, 43(2):223–243, 1942.

[14] G. Huet. Confluent reductions: Abstract properties and applications to term rewriting systems. *Journal of the ACM*, 27(4):797–821, 1980. DOI: 10.1145/322217.322230.

[15] Terese. *Term Rewriting Systems.* Cambridge University Press, 2003.

[16] C. J. Date. *An Introduction to Database Systems.* Addison-Wesley, 8th edition, 2003.

[17] J. D. Ullman. *Principles of Database and Knowledge-Base Systems, Volume I.* Computer Science Press, 1988.

[18] T. Ibaraki and T. Kameda. On the optimal nesting order for computing n-relational joins. *ACM Transactions on Database Systems*, 9(3):482–502, 1984.

[19] A. Pellenkoft, C. Galindo-Legaria, and M. Kersten. The complexity of transformation-based join enumeration. In *Proceedings of VLDB*, pages 306–315, 1996.

[20] S. J. Thomas and P. C. Fischer. Nested relational structures. *Advances in Computing Research*, 3:269–307, 1986.

[21] D. Beech. A foundation for evolution from relational to object databases. In *Proceedings of EDBT*, pages 251–270, 1988.

[22] A. Tuzhilin and J. Clifford. A temporal relational algebra as a basis for temporal relational completeness. In *Proceedings of VLDB*, pages 13–23, 1990.

[23] R. T. Snodgrass. *The TSQL2 Temporal Query Language.* Kluwer, 1995.

[24] M. Fernandez, J. Simeon, and P. Wadler. A semi-monad for semi-structured data. In *Proceedings of ICDT*, pages 263–300, 2000.

[25] P. Atzeni and V. Torlone. Management of multiple models in an extensible database design tool. In *Proceedings of EDBT*, pages 79–95, 1996.

A Proofs of Foundational Results

This appendix collects the full proof sketches of the *foundational axioms* (M1–M5, G1, G2) and the two *convergence/complexity* results (RBO Confluence and CBO complexity bound) that are stated in the body without their proofs for compactness. The four identities that are *implied* by

other operators — A7 (classical join commutativity), A8 (classical join associativity), the $\tau\,\Lambda$ / topK definitional rewrite, and the σ-τ commutation — are noted in the body but have no separate proof here. The Γ identity at depth 0 (Remark 1) is also a definitional instance and is not reproved here.

A.1 Proofs of foundational axioms

M1 — Selection commutes with materialization. Fix $r \in L$. The LHS computes $e(r)$, attaches it as c, and retains $r \oplus \{c \mapsto e(r)\}$ iff $e(r) < \tau$. The RHS evaluates $e(r) < \tau$ directly: if false, r is dropped; if true, the row is extended to $r \oplus \{c \mapsto e(r)\}$ by the μ. The two multisets are equal row-by-row. The freshness of c ensures that the predicate $c < \tau$ in LHS is unambiguous — c is defined by the μ and is not shadowed by any pre-existing attribute. ■ ■

M2 — Materialization is commutative. For each $r \in L$, LHS computes $e_1(r)$ and then $e_2(r)$, attaching both; RHS computes $e_2(r)$ and then $e_1(r)$. Because e_1 and e_2 depend only on r and not on each other's intermediate result, the two sequences yield the same extension $r \oplus \{c_1 \mapsto e_1(r), c_2 \mapsto e_2(r)\}$. The multisets are equal. ■ ■

M3 — Materialization through projection. For $r \in L$, the LHS produces $\pi_A(r) \oplus \{c \mapsto e(\pi_A(r))\}$. The RHS produces $r \oplus \{c \mapsto e(r)\}$ (using the same e since e depends only on A), then projects to $(A \cup \{c\})$. By the support of e, $e(r) = e(\pi_A(r))$, and the resulting tuples agree. The freshness of c ensures the projection is well-formed. ■ ■

A7 (join commutativity) and A8 (join associativity) are assumed classical identities; their proofs are the standard textbook one-liners and are not reproduced here.

M4 — Materialization pushdown through join. Fix a row $(r_L, r_M) \in L \bowtie_\psi M$ such that $\psi(r_L, r_M)$ holds. The LHS produces $(r_L, r_M) \oplus \{c \mapsto e(r_L)\}$. The RHS first materializes L to obtain $\mu_{e\to c}(L) = \{r'_L \oplus \{c \mapsto e(r'_L)\} : r'_L \in L\}$, then joins with M on ψ; since c is fresh and not referenced by ψ (Def. 2), the result is $\{(r'_L, r_M) \oplus \{c \mapsto e(r'_L)\} : r'_L \in L, r_M \in M, \psi(r'_L, r_M)\}$. Because e depends only on L's attributes, the materialized value $e(r_L)$ is computed in both cases from the same r_L, and ψ is unaffected by the addition of c to L's schema. The two multisets agree row-by-row. ■ ■

M5 — Materialization pushdown through graph expansion. For each row $r \in L$, the LHS first expands r via $\Gamma_{g,d}$ to obtain a multiset $\{(r, v_i) : v_i$ reachable from r.id by g within $d\}$ (multiplicity = number of paths), then materializes e on each traversal row. The result is the multiset $\{(r, v_i, c) : r \in L, v_i$ reachable from $r, c = e(r)\}$, where e depends only on r's attributes by the support condition. The RHS first materializes e on L to obtain $\mu_{e\to c}(L) = \{r \oplus \{c \mapsto e(r)\} : r \in L\}$, then expands via $\Gamma_{g,d}$. Because c is fresh and is not the graph rule's node-id attribute, the expansion proceeds from the original node-id (unchanged by the μ); the result is $\{((r, c), v_i) : r \in L, v_i$ reachable from r.id$\}$. Reordering attributes, this is $\{(r, v_i, c) : r \in L, v_i$ reachable from $r, c = e(r)\}$, the same multiset as the LHS. The two multisets agree row-by-row. ■ ■

The vector-predicate specializations V1 and J1 are A1-vector and A4-vector respectively; their proofs are the classical A1 / A4 proofs with the vector predicate substituted in, and are not reproduced here. The $\tau\,\Lambda$ / topK definitional rewrite and the σ-τ commutation are both implied by operator definitions (resp. $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L))$ *and the row-preserving semantics of σ, τ); no separate proof is required.*

G1 — Selection pushdown through graph expansion. The graph expansion $\Gamma_{g,d}$ is row-multiplicating: each origin row $r \in L$ produces one or more traversal rows in the result, and the multiplicity of r's descendants is preserved (i.e. $\Gamma_{g,d}$ does not collapse duplicates introduced by L). Let $R \subseteq L$ be the multiset of rows satisfying φ. The LHS starts from L and applies σ_φ *after* expansion, so any traversal row whose origin row lies outside R is removed; the RHS starts from R and only generates

traversal rows whose origin satisfies φ, so none are removed. The multisets agree row-by-row, because φ references only origin attributes (Def. 1 + the row-multiplicativity of Γ) — if φ referenced traversal attributes, the two sides would diverge and the identity would fail. ■ ■

G2 — Graph expansion is transitive. For each $r \in L$, the LHS expands r by g_2 to depth d_2 yielding rows (r, v_2), then expands each (r, v_2) by g_1 to depth d_1 yielding (r, v_2, v_1) for each v_1 reachable from v_2. The RHS expands r by $g_1; g_2$ to depth $d_1 + d_2$, yielding (r, v_1) for each v_1 reachable from r by the composite. By the disjoint-label condition, g_1's traversal starts at v_2 and ends at v_1, independent of r's attributes; collapsing intermediate v_2 in the LHS gives the same (r, v_1) multiset as the RHS. The non-overlap condition (g_1's traversal result disjoint from g_2's source) ensures no join is needed; without it, the LHS would require a join between g_2's targets and g_1's origins. ■ ■

A.2 Proof of RBO Confluence

RBO Confluence. Termination follows from the weight function of Section 3.5: M1, M3, M4, G1 strictly decrease w; M2, M5, G2, and the implied σ-τ reordering are zero-weight rewrites, each bounded by a finite inversion count ($k!$ materialization orderings, μ-Γ inversions, nested-Γ inversions, and σ-τ inversions respectively). The $\tau\,\Lambda$ / topK recognition is a definitional rewrite in one-to-one correspondence with its inverse, so it trivially terminates. A1-vector and A4-vector (V1, J1 in the codebase) are A1/A4 applied to vector predicates and do not produce new critical pairs beyond those of A1, A4. Local confluence holds by the critical-pair analysis of Section 3.5: the 15 critical pairs (M1∩M1, M1∩M4, M1∩G1, M1∩M5, M4∩G1, M4∩M5, M5∩G1, G1∩A4, G2∩G2, G2∩G1, G2∩M5, M2∩M1, M2∩M5, σ-τ ∩ M1, σ-τ ∩ M3) all join by a finite rewrite sequence (the support conditions transfer: σ and μ on origin-only attributes, join on L's side, etc.). By Newman's lemma, termination plus local confluence implies confluence. ■ ■

A.3 Proof of CBO complexity bound

CBO complexity bound. The CBO's search space is the product of the RBO's plan space and the per-operator physical-realization choice. From Section 4, the RBO's plan space is bounded by c^n for a constant c depending on the rule set and n operator occurrences. The CBO adds a factor of $|\text{Phys}|^n$ (at most 4 physical realizations per logical operator for the present inventory, but up to a small constant p in general). The product $|\text{Space}_{\text{RBO}}(Q)| \cdot |\text{Phys}|^n$ is exponential in n in the worst case. Standard pruning heuristics (Selinger-style dynamic programming for the $\bowtie$-fragment, greedy selectivity ordering for vector predicates, top-K pruning for $\tau\,\Lambda$) reduce the practical runtime to polynomial for the common subclasses. The bound remains exponential in the worst case; polynomial subclasses are characterized in Section 5.3. ■ ■

A.4 Reference tables

Table 1 lists the cost vectors of the core physical operators used by the CBO; the symbols are defined in the table caption.

Physical operator	Cost vector (io, cpu, mem, approx)
$F(R)$ (scan)	$(B(R),\ 0,\ B(R),\ 0)$
$B_{col,range}(R)$ (B-tree)	$(O(\log N + k),\ 0,\ k,\ 0)$
$H_{V,q,\tau}(R)$ (HNSW)	$(O(ef \log N),\ O(efd),\ O(ef),\ 1 - \text{recall}(ef, M))$
$H_{V,q,\tau}(R)$ (IVF)	$(O(nprobe \log nlist),\ O(nprobed),\ O(nprobe),\ 1 - \text{recall}(nprobe, nlist))$
$H_{V,q,\tau}(R)$ (scan)	$(O(\lvert R\rvert),\ O(\lvert R\rvert d),\ O(\lvert R\rvert),\ 0)$
$I_{col,term}$ (inverted)	$(O(p),\ O(ps),\ O(p),\ 0)$
$G_{g,d}$ (graph)	$(O(\lvert V\rvert + \lvert E\rvert),\ O(vis),\ O(vis),\ 0)$
$\text{Sort}(O)$ / $\text{TopK}(O, K)$	$(O(B \log B),\ O(B \log B),\ O(B),\ 0)$ / $(O(B \log K),\ O(B \log K),\ O(K),\ 0)$
$\text{Materialize}(e \to c)$	$(0,\ O(\lvert R\rvert \text{cost}(e)),\ O(\lvert R\rvert),\ 0)$
HashJoin / NLJ	$(B(R) + B(S),\ O(\lvert R\rvert + \lvert S\rvert),\ O(\min(\lvert R\rvert, \lvert S\rvert)),\ 0)$ / $(B(R)\lvert S\rvert,\ O(\lvert R\rvert\lvert S\rvert),\ 0,\ 0)$

Table 1. Symbolic cost vectors for core physical operators. $B(R)$ = page count of R; N = cardinality; p = postings-list length; s = per-posting score cost; ef, M = HNSW parameters; $nprobe, nlist$ = IVF parameters; B = sort / top-K batch size; vis = visited nodes in a graph walk.